

BareTorch: Challenging State-of-the-Art Sequence Mixing Topologies via Kernel-Free, Pure GEMM-Compliant Architectures

Martin Ignacio Kovacevic Buvinic

Independent Research / BareTorch Framework Laboratory

Abstract

Modern deep learning is deeply entangled with custom hardware lock-in, where the validation of alternative sub-quadratic sequence mixers (e.g., State Space Models, Gated Linear Attention) requires writing highly complex, unportable CUDA or Triton kernels to achieve operational wall-clock competitiveness. This architectural dependency acts as a major structural barrier to next-generation model exploration. In this work, we introduce the foundational principles of BareTorch, an open-source development ecosystem designed to build, scale, and evaluate competitive sequence architectures utilizing strictly pure, high-level PyTorch matrix multiplication (GEMM) equations. We formulate and evaluate five custom sub-quadratic topologies designed under this kernel-free paradigm: Chunkwise Cascaded Resonance Sifter (CCRS), Chunk-Segmented Low-Rank Associative Delta Engine (CS-LRAD), Causal Kernel-Gated Tensor Sifter (CKTS), Causal Orthogonal Feedback Engine (COFE), and the Causal Block-Kronecker Cascade (CBKC). We benchmark these systems against a broad array of state-of-the-art, kernel-dependent topologies (GLA, Mamba3, GDN2) and a production-grade Transformer baseline enhanced with Rotary Position Embeddings (RoPE), Grouped-Query Attention (GQA), and native FlashAttention. Through a systematic pretraining regimen spanning Tiny (34M), Small (100M), and Base (200M) parameter classes over a 4.2-billion token runway, we demonstrate that our kernel-free chunk-parallel configurations maintain absolute linear $O(N)$ VRAM scaling and excellent hardware throughput out to 8,192 tokens on commodity hardware, successfully bypassing the quadratic execution degradation of traditional attention and the portability constraints of custom hardware kernels.

1 Introduction

The field of foundational language modeling is currently undergoing a structural pivot away from standard $O(N^2)$ Softmax attention and toward sub-quadratic sequence-mixing mechanics. Architectural innovations such as Gated Linear Attention (GLA), Mamba variants, and Gated Delta Net-

works (GDN) have proven that hidden state recurrence can operate with linear computational and spatial complexity during decoding.

However, an unaddressed systemic bottleneck plagues this architectural evolution: the custom kernel lock-in. Under standard execution pipelines, an alternative sequence mixer expressed in high-level primitive equations (such as element-wise scans or material loops) faces severe execution degradation due to the sequential nature of PyTorch’s eager execution model. To bypass this, researchers routinely rely on custom CUDA or Triton kernel engineering. While this approach recovers hardware utilization, it restricts architectural development to specific GPU targets, significantly limits compilation portability, and prevents multi-hardware execution on alternative accelerators (e.g., TPU, WebGPU, Apple Silicon, or commodity edge devices).

To address this challenge, this paper introduces BareTorch, an engineering paradigm and architecture library built on a core philosophy: next-generation sequence-mixing topologies should challenge state-of-the-art foundation models while remaining entirely compliant with high-level, native PyTorch tensor blocks. Rather than relying on hardware-specific low-level optimization tricks, BareTorch achieves sub-quadratic execution scaling by leveraging structured block-parallel chunk segmentation. By structuring sequence hidden states into uniform, statically bound tensor blocks, the underlying computations map directly into highly optimized BLAS/GEMM routines natively accelerated by all modern hardware backends.

2 Mathematical Foundations

The primary objective of BareTorch is to replace fine-grained, token-level sequential dependencies with macro-level, block-parallel tensor operations. We implement five distinct mathematical paradigms that satisfy this constraint, allowing highly complex, recurrent, or hierarchical interactions to resolve purely via parallelized GEMM paths.

2.1 Causal Block-Kronecker Cascade (CBKC)

The Causal Block-Kronecker Cascade (CBKC) operates as an inherently sub-quadratic sequence mixer by restructuring the historical sequence timeline into a multi-scale binary tree. Over a sequence dimension L and head tracking dimension d_h , the model initializes an allocation of $S = 11$ independent stages. At each stage $s \in \{0, 1, \dots, S-1\}$, the context length is partitioned into static blocks of size $B_s = 2^{s+1}$, which are further bisected into past and future sub-blocks of size $M_s = 2^s$. Let the head tensor projections be $Q, K, V \in \mathbb{R}^{B \times H \times L \times d_h}$. At stage s , the tensors are reshaped into block-parallel sub-spaces:

$$Q_{b,s}, K_{b,s}, V_{b,s} \in \mathbb{R}^{B \times H \times \frac{L}{B_s} \times 2 \times M_s \times d_h} \quad (1)$$

The historical residue is accumulated exclusively via cross-block multiplication between the past slice ($idx = 0$) and the future slice ($idx = 1$):

$$S_{b,s} = (K_{b,s}[:, :, :, 0, :, :])^T V_{b,s}[:, :, :, 0, :, :] \quad (2)$$

The updates to the future tokens are subsequently synthesized via a direct parallel projection through this historical state matrix, modulated by a multi-scale data-dependent Kronecker gate network g_s :

$$Y_{b,s,\text{future}} = \frac{1}{\sqrt{M_s \cdot d_h}} (Q_{b,s}[:, :, :, 1, :, :] \cdot S_{b,s}) \odot g_s[:, :, :, 1, :, :] \quad (3)$$

The updates across all independent scales are mapped back to sequence dimensions and stabilized using a hierarchical tier-dependent variance normalizer:

$$Y = V + \left(\left(\sum_{s=0}^{S-1} Y_{s,\text{update}} \right) \odot \frac{1}{\sqrt{S}} \right) \quad (4)$$

2.2 Chunkwise Cascaded Resonance Sifter (CCRS)

The Chunkwise Cascaded Resonance Sifter (CCRS) frames sequence tracking as a coupled, second-order linear ordinary differential system unrolled globally in parallel via lower-triangular matrix operator composition. The total timeline L is segmented into non-overlapping blocks of size $C = 32$, yielding $N = L/C$ macro-chunks. Two data-driven damping fields (g_1, g_2) act as continuous resonance constraints, bounded via clamping to maintain numeric stability:

$$g_1 = \text{clamp}(\sigma(W_{g1}x), 10^{-3}, 0.999) \quad (5)$$

$$g_2 = \text{clamp}(\sigma(W_{g2}x), 10^{-3}, 0.999) \quad (6)$$

Within an isolated chunk, local decay is resolved by evaluating parallel cumulative operations in the log domain:

$$\Lambda_1 = \text{cumsum}(\log(g_1)), \quad \Lambda_2 = \text{cumsum}(\log(g_2)) \quad (7)$$

The local activation matrix M_{cascaded} represents the cascaded composition of both decay fields:

$$M_{\text{cascaded}} = \exp(\Lambda_2 - \Lambda_2^T) \cdot \exp(\Lambda_1 - \Lambda_1^T) \odot M_{\text{causal}} \quad (8)$$

$$Y_{\text{local}} = \left(\frac{1}{\sqrt{d_h}} (Q \cdot K^T) \odot M_{\text{cascaded}} \right) \cdot V \quad (9)$$

Cross-chunk global tracking maps historical residues across macro-blocks via matrix exponents M_{c1} and M_{c2} :

$$S_{\text{delta}} = (K \odot \exp(\Lambda_{1,\text{end}} - \Lambda_1))^T \cdot V \quad (10)$$

$$S_{\text{historical}} = (\exp(\Lambda_{c1} - \Lambda_{c1}^T - \log(g_{c1})^T) \cdot \exp(\Lambda_{c2} - \Lambda_{c2}^T - \log(g_{c2})^T)) \quad (11)$$

$$Y_{\text{global}} = \frac{1}{\sqrt{d_h}} (Q \odot \exp(\Lambda_2)) \cdot S_{\text{historical}} \quad (12)$$

2.3 Causal Kernel-Gated Tensor Sifter (CKTS)

The Causal Kernel-Gated Tensor Sifter (CKTS) introduces a bifurcated processing pipeline designed to fuse high-expressivity local chunkwise causal attention with an anisotropic, linear global state-space track.

Track 1 (Local): Resolves localized context inside chunks using non-linear Softmax attention mappings:

$$\text{Scores} = \frac{Q_l \cdot K_l^T}{\sqrt{d_h}} \odot M_{\text{causal}} \quad (13)$$

$$Y_{\text{local}} = \text{Softmax}(\text{Scores}) \cdot V_l \quad (14)$$

Track 2 (Global): Handles macro-scale linear chunk-parallel recurrence tracks. Let α be an adaptive state control gate:

$$\Lambda_\alpha = \text{cumsum}(\log(\alpha)) \quad (15)$$

$$Q_{g,\text{scaled}} = Q_g \odot \exp(\Lambda_\alpha), \quad K_{g,\text{scaled}} = \frac{K_g}{\exp(\Lambda_\alpha) + 10^{-6}} \quad (16)$$

$$S_{\text{all}} = \text{cumsum} \left(\frac{\exp(\Lambda_{\alpha,\text{end}}) \otimes (K_{g,\text{scaled}}^T \cdot V_{g,\text{gated}})}{\Sigma_{\text{chunk}} + 10^{-6}} \right) \cdot \Sigma_{\text{chunk}} \quad (17)$$

The tracks are dynamically combined through an independent element-wise feature selection gate:

$$\text{Out} = Y_{\text{local}} + (Y_{\text{global}} \odot \sigma(W_{\text{gate}}x)) \quad (18)$$

2.4 Causal Orthogonal Feedback Engine (COFE)

The Causal Orthogonal Feedback Engine (COFE) introduces an unrolled least-squares error-correction step to simulate the dynamic memory-wiping performance of Gated Delta Networks entirely through parallel chunk blocks. Before historical matrix projection, local values V are modified by subtracting their projection onto the localized key space:

$$G = \frac{1}{\sqrt{d_h}} (K \cdot K^T) \quad (19)$$

$$V_{\text{corrected}} = V - ((G \odot \mathcal{M}_{\text{causal}, \Delta=-1}) \cdot V) \odot \beta \quad (20)$$

The output tracking then processes the corrected feature state matrices in parallel, passing the residuals to the cross-chunk tracking equations:

$$Y_{\text{local}} = \left(\frac{1}{\sqrt{d_h}} (Q \cdot K^T) \odot \mathcal{M}_{\text{causal}, \Delta=-1} \right) \cdot V_{\text{corrected}} \quad (21)$$

$$S_{\text{all}} = \text{cumsum} \left(\frac{\exp(\Lambda_{\alpha, \text{end}}) \otimes (K_g^T \cdot V_g)}{\Sigma_{\text{chunk}} + 10^{-6}} \right) \cdot \Sigma_{\text{chunk}} \quad (22)$$

2.5 Chunk-Segmented Low-Rank Delta Engine (CS-LRAD)

The CS-LRAD architecture implements outer-product updates through bounded rank- r projection sub-spaces ($r = 8$). The parallel state tracking formulation unrolls the matrix transformations smoothly across blocks:

$$M_{\text{links}} = \exp(\Lambda - \Lambda^T) \odot \mathcal{M}_{\text{causal}} \quad (23)$$

$$Y_{\text{local}} = \left(\frac{1}{\sqrt{d_h}} (Q \cdot K^T) \odot M_{\text{links}} \right) \cdot V \quad (24)$$

The global cross-chunk feedback track collects historical information via low-rank bottleneck tracking:

$$U_{\text{decayed}} = (U \odot \beta_{\text{gate}}) \odot \left(\frac{\exp(\Lambda_{\text{end}})}{\exp(\Lambda) + 10^{-6}} \right) \quad (25)$$

$$S_{\text{historical}} = M_{\text{chunks}} \cdot (U_{\text{decayed}}^T \cdot V) \quad (26)$$

$$Y_{\text{global}} = \frac{1}{\sqrt{d_h}} (R \odot \exp(\Lambda)) \cdot S_{\text{historical}} \quad (27)$$

3 Experimental Configuration Setup

To ensure absolute empirical isolation, all architectures were compiled from identical structural baselines using the BareTorch factory layer engine. Optimization scaling parameters are balanced across short-context and long-context windows by scaling hardware mini-batch dimensions dynamically. Table 1 catalogs the absolute execution parameters and configuration overrides driving the pretraining runway.

4 Experimental Results

4.1 Convergence Analysis

Table 2 maps the final convergence metrics recorded at the 30,000-step anchor mark across all evaluated model configurations.

4.2 Performance-Portability Pareto Frontier

The empirical results confirm that BareTorch chunk topologies break the traditional performance-portability trade-off. At the 200M Base tier, `cs_lrad_base_2048` achieves a validation perplexity of 18.92. This directly targets the performance of the production-grade Transformer baseline (17.43), which relies on highly optimized FlashAttention tracking blocks. More importantly, CS-LRAD completes its training runway in 27,223.50 seconds, natively outspeeding the FlashAttention-powered Transformer (27,937.40 seconds) and the hardware-fused custom Triton kernel of GDN2 (29,983.23 seconds). This proves that block-parallel chunk structures can match or exceed the wall-clock velocity of hardware-specific optimizations.

Additionally, the Causal Block-Kronecker Cascade (`cbkc_base_2048`) demonstrates powerful representation capacity, achieving a validation perplexity of 18.64 at the Base tier. This narrows the modeling gap to the production-grade SOTA Transformer to a merge 1.21 perplexity points, confirming that pure high-level GEMM blocks can match complex recurrence without low-level fusion scripts.

4.3 Generation Throughput Profiles

To measure runtime performance during deployment, sequential step generation throughput was evaluated. Table 3 details the token processing velocity across scaling thresholds.

5 Discussion & Conclusion

The empirical results demonstrate an essential realization: the Performance-Portability Pareto Frontier. While Gated Delta Networks (GDN2) achieve robust cross-entropy ceilings, they are strictly dependent on specialized Triton low-level compilation workflows. Attempting to deploy them on non-CUDA infrastructure leads to immediate throughput degradation. In contrast, the BareTorch family—most notably CS-LRAD and CBKC—demonstrates that highly parallelized block-parallel formulations can match or exceed both the convergence and execution speed of customized kernels. By structuring global history updates into pure PyTorch GEMM blocks, CS-LRAD matches the modeling capacity of a highly optimized SOTA Transformer while delivering a 10.5% generation speed advantage at the Base tier. This confirms that BareTorch serves as a zero-cost abstraction, delivering performance portability across arbitrary modern compute environments.

t

Table 1: Comprehensive Pretraining Hyperparameter & Run Ledger

Configuration ID	Target Mixer	Len	Batch	Accum	Dim	Layers	Heads	Chkpt	Chunk Spec
transformer_tiny_256	SOTA Transformer	256	64	2	256	8	16	False	—
transformer_tiny_2048	SOTA Transformer	2048	8	2	256	8	16	False	—
transformer_small_2048	SOTA Transformer	2048	8	2	512	12	16	False	—
transformer_base_2048	SOTA Transformer	2048	8	2	768	16	16	True	—
mamba3_tiny_256	Mamba3	256	64	2	256	8	4	False	—
mamba3_tiny_2048	Mamba3	2048	8	2	256	8	4	False	—
mamba3_small_2048	Mamba3	2048	8	2	512	8	8	False	lr 8e-5
gla_tiny_256	GLA	256	64	2	256	8	4	False	—
gla_tiny_2048	GLA	2048	8	2	256	8	4	False	—
gla_small_2048	GLA	2048	8	2	512	12	8	True	—
gdn2_tiny_256	Gated Delta Net 2	256	64	2	256	8	4	False	—
gdn2_tiny_2048	Gated Delta Net 2	2048	8	2	256	8	4	False	—
gdn2_small_2048	Gated Delta Net 2	2048	8	2	512	8	8	True	—
gdn2_base_2048	Gated Delta Net 2	2048	8	2	768	10	10	True	—
cs_lrad_tiny_256	CS-LRAD	256	64	2	256	8	16	False	C32 (R8)
cs_lrad_tiny_2048	CS-LRAD	2048	8	2	256	8	16	False	C32 (R8)
cs_lrad_small_2048	CS-LRAD	2048	8	2	512	12	16	False	C32 (R8)
cs_lrad_base_2048	CS-LRAD	2048	8	2	768	14	16	True	C32 (R8)
cofe_tiny_256	COFE	256	64	2	256	8	16	False	C32
cofe_tiny_2048	COFE	2048	8	2	256	8	16	False	C32
cofe_small_2048	COFE	2048	8	2	512	12	12	True	C32
ckts_tiny_256	CKTS	256	64	2	256	8	16	False	C32
ckts_tiny_2048	CKTS	2048	8	2	256	8	16	False	C32
ckts_small_2048	CKTS	2048	8	2	512	12	12	True	C32
ccrs_tiny_256	CCRS	256	64	2	256	8	16	False	C32
ccrs_tiny_2048	CCRS	2048	8	2	256	8	16	False	C32
ccrs_small_2048	CCRS	2048	8	2	512	12	8	True	C32
cbkc_tiny_256	CBKC	256	64	2	256	8	16	False	C32
cbkc_tiny_2048	CBKC	2048	8	2	256	8	16	False	C32
cbkc_small_2048	CBKC	2048	8	2	512	12	16	True	C32
cbkc_base_2048	CBKC	2048	8	2	768	14	16	True	C32

Table 2: Pretraining Validation Perplexity Ledger

Configuration ID	Params	Loss	Val PPL	Time (s)
Short Context (L=256) Configurations				
transformer_tiny_256	34.39M	3.7868	44.11	5,012.44
gla_tiny_256	33.39M	3.7279	41.59	4,988.83
mamba3_tiny_256	35.04M	3.6342	37.87	5,457.17
gdn2_tiny_256	39.16M	3.4591	31.79	7,369.64
cs_lrad_tiny_256	34.45M	3.8183	45.53	5,492.64
ccrs_tiny_256	33.40M	3.9816	53.60	5,602.43
Long Context (L=2048) Configurations				
transformer_tiny_2048	32.55M	3.7029	40.57	5,043.19
gla_tiny_2048	33.39M	3.6372	37.99	4,888.46
mamba3_tiny_2048	35.04M	3.7470	42.39	5,646.63
gdn2_tiny_2048	39.16M	3.3068	27.30	7,244.08
cs_lrad_tiny_2048	34.45M	3.7096	40.84	5,507.01
ccrs_tiny_2048	33.40M	3.9489	51.88	5,638.36
cbkc_tiny_2048	33.70M	3.7779	43.72	8,010.59
cofe_tiny_2048	33.93M	3.8994	49.37	5,968.18
ckts_tiny_2048	35.47M	3.7630	43.08	6,357.22
transformer_small_2048	92.37M	3.1375	23.05	11,094.79
gla_small_2048	97.24M	3.1461	23.25	12,543.51
gdn2_small_2048	101.92M	2.9355	18.83	16,696.98
cs_lrad_small_2048	102.00M	3.1815	24.08	12,342.54
ccrs_small_2048	97.19M	3.3685	29.04	14,306.86
cbkc_small_2048	98.18M	3.1789	24.02	26,394.23
ckts_small_2048	109.36M	3.1660	23.71	20,890.10
cofe_small_2048	100.15M	3.2059	24.68	19,007.20
mamba3_small_2048*	87.49M	4.1292	62.13	9,663.16
transformer_base_2048	199.90M	2.8580	17.43	27,937.40
gdn2_base_2048 "	203.55M	2.7451	15.57	29,983.23
cs_lrad_base_2048	208.31M	2.9401	18.92	27,223.50
cbkc_base_2048	198.87M	2.9252	18.64	54,266.66

Table 3: Step Inference Generation Velocity (Tokens/Second)

Core Architecture Type	Tiny	Small	Base
TRANSFORMER_SOTA (RoPE+GQA+Flash)	166.57	113.37	87.02
GLA (Kernel-Fused)	117.80	78.28	—
MAMBA3 (Kernel-Fused)	103.12	90.98	—
GDN2 (Kernel-Fused)	75.16	73.69	53.66
CCRS (BareTorch)	202.70	140.62	—
CS_LRAD (BareTorch)	164.97	113.41	96.23
CBKC (BareTorch)	56.77	37.29	29.35